

AEGIS IDEA 3

Cyber-Physical Lockdown Controller

สรุปสิ่งที่ดำเนินการ: Build / Upload เฟิร์มแวร์ ESP32
และทดสอบระบบความปลอดภัย MQTT ทั้ง 5 สถานการณ์

วันที่: 22 กรกฎาคม 2569 (2026-07-22)

บอร์ด: ESP32-D0WD-V3 (MAC b4:bf:e9:33:0c:7c)

พอร์ต: /dev/ttyUSB1 (CP210x)

1. ภาพรวมสิ่งที่ทำวันนี้

เซสชันนี้ดำเนินการ 3 ขั้นตอนหลักกับโปรเจกต์ AEGIS IDEA 3 (ระบบตัดสาย Uplink ของ NAS ผ่าน ESP32 + Relay เมื่อตรวจพบภัยคุกคาม หรือเมื่อ NAS ขาดการติดต่อ) ดังนี้

- Build และ Upload เฟิร์มแวร์ (src/main.cpp) ลงบอร์ด ESP32 จริงผ่าน PlatformIO
- เปิด Serial Monitor เพื่อยืนยันว่าบอร์ดบูตขึ้นมาทำงานถูกต้อง ต่อ WiFi และ MQTT broker สำเร็จ
- ทดสอบยิงคำสั่งผ่าน MQTT ครบทั้ง 5 สถานการณ์ตามแผนทดสอบของโปรเจกต์ (Valid / Replay / Tamper / Stale / Dead Man's Switch) แล้วตรวจผล ACK จริงจากบอร์ด

สรุปผลลัพธ์โดยย่อ: ระบบผ่าน 4 ใน 5 สถานการณ์ตามที่ออกแบบไว้ และพบข้อสังเกต 1 จุดที่ควรแก้ไขก่อนใช้งานจริง (ดูหัวข้อ 4 และ 5)

2. Build และ Upload เฟิร์มแวร์

2.1 ปัญหาที่พบ

platformio.ini เดิมตั้ง upload_port/monitor_port ไว้ที่ /dev/ttyUSB0 แต่เมื่อตรวจสอบ USB จริง (lsusb) พบชิป Silicon Labs CP210x UART Bridge ของ ESP32 ขึ้นเป็น /dev/ttyUSB1 แทน นอกจากนี้ค่า upload_speed เดิมตั้งไว้ที่ 921600 ซึ่งตามบันทึกของทิม (CLAUDE.md) เคยทำให้ verify fail มาก่อน

2.2 การแก้ไข

แก้ไขไฟล์ platformio.ini ดังนี้ (ตามที่ผู้ใช้อยืนยันให้ใช้ค่า 115200 ตามบันทึกเดิม):

```
monitor_speed = 115200
upload_speed   = 115200 ; 921600 -> CLAUDE.md verify fail
upload_port    = /dev/ttyUSB1
monitor_port   = /dev/ttyUSB1
```

2.3 ผลลัพธ์

Build: **SUCCESS**

RAM ใช้ไป: 13.8% (45,260 / 327,680 bytes)

Flash ใช้ไป: 58.2% (762,969 / 1,310,720 bytes)

Upload: **SUCCESS**

ชิป: ESP32-D0WD-V3 (revision v3.1)

MAC Address: b4:bf:e9:33:0c:7c

Flash Hash: Verified ครอบคลุม (bootloader / partitions / firmware)

: warning "Detected crystal freq 15.55MHz is quite different to normalized freq 26MHz" esptool auto-detect 26MHz upload

3. ตรวจสอบผ่าน Serial Monitor

การเปิด Serial Monitor แบบ interactive (pio device monitor) ใช้งานไม่ได้ในสภาพแวดล้อมนี้ เนื่องจากต้องการ tty จริงสำหรับรับคีย์บอร์ด จึงเปลี่ยนไปใช้สคริปต์ pyserial อ่านข้อมูลดิบจากพอร์ตแทน

3.1 อุปสรรคที่พบระหว่างทาง

- ความพยายามแรกอ่าน serial ค้างนานผิดปกติ (2 นาทีกว่า จากที่ตั้งไว้ 15 วิ) และกิน CPU สูง — สาเหตุคือการเปิดพอร์ตแบบ pyserial ปกติไปกระตุ้นขา DTR/RTS ทำให้บอร์ด ESP32 รีเซ็ตวนซ้ำ ๆ (เห็นจาก log "rst:ets ..." ซ้ำหลายสิบครั้ง)
- kill process DTR/RTS (setDTR(False), setRTS(False)) timeout — serial port

3.2 Log ที่อ่านได้หลังบอร์ดบูตนิ่ง

```
=====
AEGIS IDEA 3 - Secure MQTT Lockdown
=====
WiFi: Pboo_2.4G
WiFi OK! IP: 192.168.2.195
MQTT broker...
subscribe: aegis/lockdown/cmd
=====
```

ยืนยันว่าเฟิร์มแวร์ทำงานถูกต้องครบ 3 ขั้นตอน: บูต → ต่อ WiFi (2.4GHz) สำเร็จ → ต่อ MQTT broker และ subscribe topic สำเร็จ

ต่อมาตรวจสอบผ่าน mosquitto broker (รันอยู่บนเครื่องนี้เอง ที่ 192.168.1.167:1883) พบว่า ESP32 เชื่อมต่ออยู่จริงแบบ ESTABLISHED (เห็นจาก ss -tn และ journalctl ของ mosquitto) ยืนยันว่าบอร์ดพร้อมรับคำสั่งจริง โดยการต่อ WiFi นี้เป็นเพียงช่องทางสั่งการ (MQTT command channel) เท่านั้น ไม่เกี่ยวกับสาย Uplink ที่ยังไม่ได้ต่อจริง

4. ทดสอบคำสั่ง MQTT ทั้ง 5 สถานการณ์

ยังคำสั่งไปยัง topic aegis/lockdown/cmd (และ aegis/heartbeat) ด้วยสคริปต์ Python ที่คำนวณ HMAC-SHA256 เอง (คีย์ลับตรงกับ HMAC_SECRET ในเฟิร์มแวร์) ผ่าน mosquitto_pub และดึงฟังผลลัพธ์จริงจาก topic aegis/lockdown/ack และ aegis/status ด้วย mosquitto_sub

Test 1: Valid CUT_UPLINK ผ่านตามแบบ

เหตุผลที่ส่ง: ส่งคำสั่งตัด uplink ที่มี HMAC / nonce / timestamp ถูกต้องครบทุกอย่าง เพื่อพิสูจน์ว่า path การทำงานปกติ (Happy Path) ใช้งานได้จริง

```
: { "cmd": "CUT_UPLINK", "nonce": "n1-...", "ts": "...", "sig": "..."}
: ACK: {"ack": "OK", "detail": "uplink cut"} | status: LOCKDOWN
```

ตรงตามที่ออกแบบไว้ — ระบบรับคำสั่งและตัด uplink สำเร็จ

Test 2: Replay Attack ผ่านตามแบบ

เหตุผลที่ส่ง: ส่งข้อความชุดเดิมจาก Test 1 ซ้ำอีกครั้งทุกไบต์ (nonce เดิม) เพื่อพิสูจน์กลไกกัน Replay Attack ตาม Failure Case #5 ของเอกสารโครงการ

```
: ( Test 1 )
: ACK: {"ack": "REPLAY", "detail": "nonce reused"}
```

ตรงตามที่ออกแบบไว้ — nonce ที่ใช้ไปแล้วถูกปฏิเสธ ป้องกัน Replay ได้จริง

Test 3: Tamper Attack ผ่านตามแบบ

เหตุผลที่ส่ง: เซ็น HMAC ไว้สำหรับคำสั่ง RESTORE_UPLINK (nonce/ts ชุดใหม่) แล้วแอบแก้ฟิลด์ cmd เป็น CUT_UPLINK ก่อนส่งจริง เพื่อพิสูจน์ว่าการแก้ไข payload หลังเซ็นแล้วจะถูกจับได้ (Integrity)

```
: { "cmd": "CUT_UPLINK", ... , "sig": "<sig RESTORE_UPLINK>" }
: ACK: {"ack": "BAD_HMAC", "detail": "sig mismatch"}
```

ตรงตามที่ออกแบบไว้ — ลายเซ็นไม่ตรงกับเนื้อหาจริง ถูกปฏิเสธถูกต้อง

Test 4: Stale Timestamp พบข้อสังเกต

เหตุผลที่ส่ง: ส่งคำสั่งที่ signature ถูกต้อง 100% แต่ timestamp เก่ามาก (ts=1000000000 ปี ค.ศ. 2001) เพื่อตรวจสอบว่าเฟิร์มแวร์ปฏิเสธคำสั่งที่หมดอายุตามที่ comment ในโค้ดอ้างไว้ ("เก่าเกิน 30 ปี = ปฏิเสธ") หรือไม่

```
: { "cmd": "CUT_UPLINK", "ts": 1000000000, "sig": "<sig ts >" }
: ACK: {"ack": "OK", "detail": "uplink cut"} <-- "STALE"
```

ไม่ตรงตามที่ออกแบบไว้ — ระบบยอมรับคำสั่งที่ ts เก่ามากราวกับเป็นคำสั่งใหม่ (ดูรายละเอียดช่องโหว่ในหัวข้อ 5)

Test 5: Dead Man's Switch ผ่านตามแบบ

เหตุผลที่ส่ง: ส่ง heartbeat 1 ครั้งเพื่อ "arm" ตัวจับเวลาในเฟิร์มแวร์ จากนั้นหยุดส่งไปเลยและรอ 65 วินาที (เกิน DEADMAN_TIMEOUT_MS=60000 ที่ตั้งไว้) เพื่อพิสูจน์ Failure Case #6: ถ้า NAS ถูกยึด/หยุดทำงานจนไม่มี heartbeat ระบบต้องตัด uplink เองโดยอัตโนมัติ

```
: aegis/heartbeat: "ping" ( )
: status: LOCKDOWN ( 65 )
```

ตรงตามที่ออกแบบไว้ — Dead Man's Switch ทำงานจริง ตัด uplink เองอัตโนมัติเมื่อขาด heartbeat เกินกำหนด

Cleanup

หลังครบ 5 สถานการณ์ ส่งคำสั่ง RESTORE_UPLINK (HMAC/nonce/ts ใหม่ ถูกต้องทั้งหมด) เพื่อคืนสถานะบอร์ดกลับเป็นปกติ

```
: {"ack": "OK", "detail": "uplink restored"} | status: NORMAL
```

บอร์ดกลับสู่สถานะปกติเรียบร้อยแล้ว (ไฟเขียวติด, relay ปล่อย, ไม่มีการตัดวงจรค้างไว้)

5. ข้อสังเกตสำคัญที่พบจากการทดสอบจริง

การทดสอบวันนี้เป็นการรันกับฮาร์ดแวร์จริงครั้งแรก (ต่างจาก demo_offline.py ที่เป็นแค่ simulation) ทำให้เจอจุดที่โค้ดจริงใน src/main.cpp ยังไม่ตรงกับ comment/เอกสารออกแบบไว้ 2 จุด ควรพิจารณาแก้ก่อนใช้งานจริงหรือรายงานในเอกสารโครงการ:

5.1 ไม่มีการตรวจสอบ Timestamp Window (Stale Command)

Comment บนสุดของ src/main.cpp ระบุว่า "Timestamp -> คำสั่งเก่าเกิน 30 วิ = ปฏิเสธ" แต่เมื่อไล่โค้ดจริงในฟังก์ชัน onMqttMessage() พบว่ามีการตรวจสอบเพียง 3 อย่างคือ ข้อมูลครบ / nonce ซ้ำ / HMAC ตรงหรือไม่ เท่านั้น ไม่มีเงื่อนไขเปรียบเทียบ ts กับเวลาปัจจุบันเลย ผลคือคำสั่งที่เซ็นไว้ตั้งแต่ปี 2001 (ts เก่ามาก) แต่ยังไม่เคยถูกใช้ nonce นั้นมาก่อน จะถูกยอมรับเสมือนเป็นคำสั่งใหม่ (ยืนยันจากผล Test 4 ข้างต้น) — ทำให้กลไก "ป้องกัน replay ด้วยหน้าต่างเวลา" ที่ตั้งใจออกแบบไว้ในเอกสารยังไม่ทำงานจริงในเฟิร์มแวร์ปัจจุบัน

- : (nonce)
nonce
history 20
- : if (abs((long)(time(nullptr) - ts)) > 30) {
"STALE" } HMAC — NTP sync ESP32

5.2 Heartbeat ยังไม่ได้ตรวจสอบลายเซ็น

จากโค้ดปัจจุบัน เมื่อข้อความมาที่ topic aegis/heartbeat ระบบจะรีเซ็ตตัวจับเวลา Dead Man's Switch ทันทีโดยไม่ตรวจสอบเนื้อหาหรือ HMAC ใด ๆ เลย (ต่างจากคำสั่ง CUT_UPLINK/RESTORE_UPLINK ที่ต้องผ่าน HMAC) ต่างจากที่เอกสารโครงการระบุไว้ว่า Heartbeat ควรเป็น "สัญญาณที่เซ็น HMAC"

- ผลกระทบ: ผู้ที่เข้าถึง MQTT broker ได้ (แม้ไม่รู้ HMAC secret) สามารถส่งข้อความอะไรก็ได้ไปที่ topic นี้เพื่อป้องกันไม่ให้ Dead Man's Switch ทำงาน ทั้งที่ NAS ตัวจริงอาจถูกยึดไปแล้ว — เป็นช่องโหว่ที่ขัดกับเจตนาของ Failure Case #6
- แนวทางแก้: เพิ่ม field HMAC signature ในข้อความ heartbeat เช่นเดียวกับคำสั่งอื่น แล้วตรวจสอบก่อน reset ตัวจับเวลา

หมายเหตุ: ทั้งสองจุดนี้เป็นข้อสังเกตจากพฤติกรรมจริงของเฟิร์มแวร์ที่รันบนบอร์ด ไม่ใช่ข้อบกพร่องของสถาปัตยกรรมที่ออกแบบไว้ในเอกสารโครงการ (AEGIS_System_Design.docx ระบุกลไกไว้ถูกต้องแล้ว) เพียงแต่การเขียนโค้ดยังตามไม่ทันการออกแบบในจุดนี้ — เหมาะจะเป็นประเด็น "lesson learned" จากการทดสอบจริงครั้งแรก" ที่ใส่ในรายงานความคืบหน้าได้

6. สรุปสถานะล่าสุดและขั้นตอนถัดไป

สถานะปัจจุบัน

- เฟิร์มแวร์ build/upload ผ่าน ต่อ WiFi และ MQTT broker ได้จริงบนฮาร์ดแวร์จริง (ไม่ใช่ simulation แล้ว)

- 3/4 (HMAC / Replay / Tamper / Dead Man's Switch)
- Uplink —

แนะนำขั้นตอนถัดไป

- แก้ไข src/main.cpp ให้เพิ่มการตรวจสอบ timestamp window ตามข้อ 5.1 และเพิ่ม HMAC ให้ heartbeat ตามข้อ 5.2
- Uplink LED / (Latency) /
- (5)
- "expected" "confirmed" (5.1)